

AI Stock & Options Trading Agent — Definitive Build Guide (v2, optimized)

Purpose of this document: a single, ordered, unambiguous spec that a **Replit agent (or Claude Code / Codex / Cursor)** can follow to build an LLM-driven stock & options trading agent — research -> strategy generation -> backtest -> paper/live execution — that is **stored in GitHub** and **deployed to Vercel or a VPS**. Written to be exported as a PDF and handed to a builder agent. Human-input checkpoints are marked **[HUMAN] HUMAN INPUT**.

0. Deliverables produced from this plan

1. **AI-Trader-Plan-v1.pdf** — the original strategy (saved verbatim).
 2. **AI-Trader-BuildGuide-v2.pdf** — this optimized, agent-ready build guide.
 3. (On approval) a scaffolded GitHub-ready repo `ai-trader/` matching Section 7.
- PDFs are generated from the Markdown via `pandoc` (fallback: Python `weasyprint` / `fpdf2`) and saved to the scratchpad, then delivered to you.

1. Context & the one critical correction

You want an "AI/LLM trader" that researches markets (historical charts/prices + other data), **generates and backtests algorithms**, and **executes stock + options trades**, with the best low-cost MCP integrations, buildable by a Replit agent, exportable to GitHub, deployable to Vercel/VPS.

Critical correction — TradingView is not a retail execution API. You cannot have an LLM "place trades through TradingView." The real pattern is TradingView Pine alert -> webhook -> bridge (TradersPost) -> broker. This plan executes **directly via the broker API (Alpaca)** and treats TradingView as an **optional signal source** (webhook in), not the executor.

Honest framing (read before building)

- LLMs have **no inherent price-prediction edge**. Their value is **orchestration, research synthesis, strategy generation, and risk reasoning** layered on top of **deterministic, backtested** strategies. Do not treat the model as an oracle.
- **Backtests ≠ live results** (overfitting, slippage, look-ahead bias, regime change). Options can go to zero fast.
- **Mandatory: paper-trade for weeks before any real capital.** Hard risk limits live in **code**, never only in prompts. (`ai-hedge-fund` itself ships an explicit "educational only, do not point at real money without extensive testing" disclaimer — we adopt the same stance.)

2. What we borrow from the top open-source projects (2026)

Project		What we take
TradingAgents (<code>TauricResearch/TradingAgents</code>)	~10.5K	Core architecture: LangGraph "trading firm" — Analyst(s) -> Bull/Bear researchers (debate) -> Trader -> Risk team -> execution. Modular, multi-LLM.
	~49.6K	Investor-persona committee (Buffett/Munger/Graham/Burry/Ackman/Wood/Damodaran) + Technicals + Risk Manager + Portfolio Manager

Project		What we take
ai-hedge-fund (viratth/ai-hedge-fund)		makes the final call. Financial Datasets API for fundamentals. Poetry/CLI + web UI pattern.
NoFx (NoFxAiOS/nofx)	~11.2K	Safety Mode (auto-halt after N consecutive losses = regime-change circuit breaker) + model-competition mode (e.g., Claude vs DeepSeek on segregated paper accounts, separate decision logs). [!] NoFx shipped real auth CVEs — we implement the ideas but with proper auth and no public no-auth endpoints.
HKUDS/AI-Trader (HKUDS/AI-Trader)	—	Agent-native platform layout: FastAPI backend + separate background workers , SQLite->Postgres, skill registration (SKILL.md) so any agent can join, \$100K paper accounts , multi-asset incl. options, optional Polymarket prediction-market paper trading. We mirror the backend-worker split and skill-doc idea.
FinRL / FinRL-Trading	—	Optional advanced module: deep-RL strategy generation, plugged in behind the same Strategy interface.
awesome-ai-in-finance, prediction-market-analysis	—	Curated reference list; optional Polymarket datasets for a prediction-market sub-agent.

Net design: TradingAgents' LangGraph firm structure, **enriched** with ai-hedge-fund's persona committee + Portfolio Manager, **guarded** by NoFx-style Safety Mode, **packaged** in HKUDS/AI-Trader's FastAPI-backend + worker + skill-doc layout, with FinRL as an optional plug-in.

3. Recommended low-cost stack (definitive)

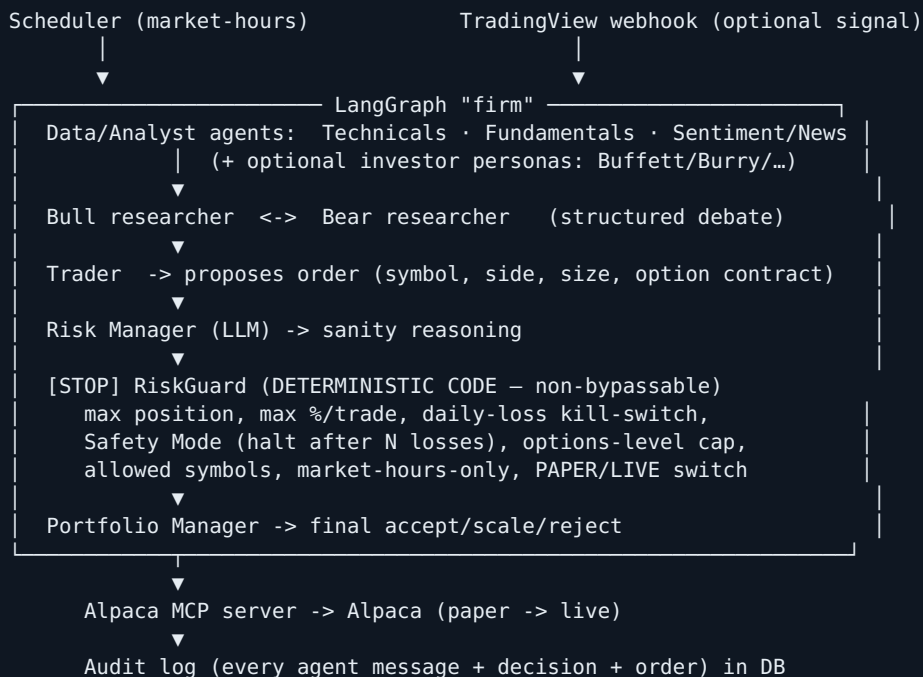
Layer	Choice	Cost
Runtime	Python 3.11	—
Agent framework	LangGraph (+ langchain-mcp-adapters)	free
LLM	Claude via Anthropic API — Haiku 4.5 for high-volume analyst/loops, Sonnet 4.6 for Trader/Risk/Portfolio nodes, Opus for periodic deep research. Prompt caching + Batch API to cut cost. Optional model-competition second model (DeepSeek/Qwen) on a segregated paper account.	~\$5-40/mo
Broker / execution	Alpaca (stocks + options, free paper, live commission-free)	\$0
Execution MCP	alpaca-hq/alpaca-mcp-server (official v2, ~65 tools, paper default)	free
Price data	Alpaca Market Data (bars/quotes) + Polygon.io Free->Starter for deeper/options history	\$0-29
Fundamentals/news	Financial Datasets API (as ai-hedge-fund uses) + a web-search MCP	\$0-low
Backtesting	VectorBT (fast research/param sweeps) + Backtesting.py (sanity cross-check); QuantLib for option greeks; NautilusTrader later for backtest=live parity	free
Scheduler	APScheduler (in-proc) or host cron/worker	free
Storage	SQLite -> Postgres	\$0-low
Backend	FastAPI (API) + separate worker process (HKUDS pattern)	—
Dashboard	Next.js on Vercel (read-only)	\$0
Always-on host	VPS (Hetzner/DO ~\$5-6/mo) or Railway hobby	~\$5
Source control	GitHub (private)	\$0

Hosting decision (Vercel and VPS — how they split)

- **Vercel** = **stateless dashboard + webhook receiver only**. Serverless functions are ephemeral and time-limited (≤ 14 min) with no persistent scheduler — **they cannot run the trading loop**.
- **VPS / Railway** = the **always-on trading worker + scheduler + agent graph**. This is the brain and must run 24/7.
- **GitHub** stores **all code + this plan**; both Vercel and the VPS deploy from GitHub (Vercel via Git integration; VPS via `git pull` + Docker, or a GitHub Actions deploy).

```
GitHub (code + plan)
├─> Vercel : Next.js dashboard + /webhook (read-only, stateless)
├─> VPS/Railway : FastAPI API + worker + LangGraph agents + scheduler
│                  │
│                  └─ Alpaca MCP server (paper->live)
│                     └─ Data (Alpaca, Polygon, Financial Datasets)
│                        └─ SQLite/Postgres audit log
```

4. Target architecture (the agent graph)



The **RiskGuard** and **Safety Mode** are plain code the LLM cannot talk its way past — this is the most important safety property in the whole system.

5. MCP setups & integrations (the "best" low-cost set)

1. **Alpaca MCP (primary — execution + account + option chains)**. `git clone https://github.com/alpacahq/alpaca-mcp-server`; env `ALPACA_API_KEY`, `ALPACA_SECRET_KEY`, `ALPACA_PAPER_TRADE=true`. Run as stdio/Docker MCP server; the Trader node calls it via `langchain-mcp-adapters`.
2. **Data MCP (optional)**. Polygon or Financial-Datasets MCP for history/fundamentals + a **web-search MCP** for news/sentiment. Curated options: `LLMQuant/awesome-trading-agents`.
3. `mcp.json` (consumed by the LangGraph MCP client):

```
{
  "mcpServers": {
    "alpaca": {
      "command": "python",
      "args": ["-m", "alpaca_mcp_server"],
      "env": {
        "ALPACA_API_KEY": "${ALPACA_API_KEY}",
        "ALPACA_SECRET_KEY": "${ALPACA_SECRET_KEY}",
        "ALPACA_PAPER_TRADE": "true"
      }
    }
  }
}
```

1. **Hybrid with eigent (optional).** The workspace repo `eigent` is a CAMEL-AI **desktop** app that already loads MCP servers from `~/.eigent/mcp.json` (`electron/main/utils/mcpConfig.ts`) and wraps tools via `AbstractToolkit.get_tools()` (`backend/app/utils/toolkit/`). To drive the trader from eigent's chat UI, just add the Alpaca MCP server to `~/.eigent/mcp.json` — no core change. (Eigent is **not** the runtime for 24/7 execution; the VPS worker is.)

6. Build order for the Replit/builder agent (logical, ordered, with human checkpoints)

Paste this section into the Replit agent. Complete phases **in order**; do not skip the human checkpoints.

Phase A — Scaffold & secrets 1. Create Python 3.11 project `ai-trader` (use `uv`) with the layout in Section 7. 2. Add `.env.example` listing every secret/limit (Section 8). [HUMAN] **HUMAN INPUT:** put real values in **Replit Secrets** (never commit): `ANTHROPIC_API_KEY` , `ALPACA_API_KEY` , `ALPACA_SECRET_KEY` , optional `POLYGON_API_KEY` , `FINANCIAL_DATASETS_API_KEY` .

Phase B — Data layer 3. Implement data adapters: Alpaca (bars/quotes/option chains) primary; Polygon + Financial Datasets fallback. Cache to SQLite. Add a `get_history()` and `get_option_chain()` API.

Phase C — Strategies + backtest (prove edge BEFORE any agent) 4. Implement 2 deterministic baseline strategies behind a `Strategy` interface: (a) equities MA-crossover/momentum; (b) options income scan (cash-secured puts / covered calls). 5. VectorBT backtest runner -> metrics (CAGR, Sharpe, max drawdown, win rate); cross-check one in `Backtesting.py`. Persist runs. [HUMAN] **HUMAN INPUT / GATE:** review backtest metrics; approve which strategies proceed.

Phase D — RiskGuard + Safety Mode (deterministic, tested) 6. `risk/` module enforced on **every** order: max position USD, max %/trade, daily-loss kill-switch, allowed symbols, options-level cap, market-hours-only, PAPER/LIVE switch, **Safety Mode** (halt after N consecutive losing trades — NoFx idea). 7. Unit tests must prove it **rejects** oversized / over-loss / off-hours / disallowed orders.

Phase E — Agent graph (LangGraph) 8. Build nodes: Technicals, Fundamentals, Sentiment analysts -> Bull/Bear debate -> Trader -> Risk Manager (LLM) -> **RiskGuard (code)** -> Portfolio Manager (final). Wire LLM tiers (Haiku analysts, Sonnet trader/risk). Optional investor personas (ai-hedge-fund style) behind a flag. 9. Connect Trader to the **Alpaca MCP** server; **paper orders only** while `TRADING_MODE=paper` .

Phase F — Orchestration & API 10. FastAPI app (status, positions, decisions, manual approve/deny) + **separate worker** process running APScheduler market-hours jobs + position/stop monitoring (HKUDS split). 11. Optional `/webhook/tradingview` (shared-secret validated) feeding alerts as a signal. 12. Audit log: persist every agent message, decision, and order.

Phase G — Dashboard (Vercel, optional) 13. Next.js read-only dashboard (positions, PnL, decision log) reading the API/DB.

Phase H — Tests, docs, ship 14. `pytest` green (esp. RiskGuard). 15. `README.md` + `SKILL.md` (HKUDS-style agent registration doc) + risk warnings. 16. Push to GitHub. 17. Deploy (Section 9).

7. Repository layout (GitHub-ready)

```
ai-trader/
├── README.md           # setup/run/test/deploy + risk warnings
├── SKILL.md            # how an agent registers/uses this (HKUDS-style)
├── plan/               # this build guide + v1 plan (the PDFs' source)
├── pyproject.toml      # uv deps
├── .env.example        # all secrets/limits named, no values
├── mcp.json            # Alpaca (+ data) MCP config
├── Dockerfile
├── docker-compose.yml  # worker + api + optional Postgres
├── vercel.json         # dashboard config (Vercel)
├── src/
│   ├── config.py       # env + PAPER/LIVE master switch
│   ├── data/           # alpaca/polygon/financial-datasets adapters + cache
│   ├── strategies/     # Strategy interface + baselines (+ optional FinRL)
│   ├── backtest/       # vectorbt + backtesting.py runners
│   ├── risk/           # RiskGuard + Safety Mode (DETERMINISTIC) + tests
│   ├── agents/         # LangGraph graph: analysts, bull/bear, trader, risk, PM, personas
│   ├── execution/      # Alpaca MCP client wrapper
│   ├── api/            # FastAPI (status, approve/deny, webhook)
│   ├── worker.py       # APScheduler market-hours loop
│   └── main.py         # entrypoint
├── dashboard/         # Next.js (Vercel)
├── tests/              # test_risk.py (critical), test_strategy.py, test_backtest.py,
└── test_execution_paper.py
```

8. `.env.example` (named secrets & hard limits)

```
# LLM
ANTHROPIC_API_KEY=
COMPETITION_MODEL_API_KEY=          # optional 2nd model (DeepSeek/Qwen) for model-competition
# Broker (Alpaca)
ALPACA_API_KEY=
ALPACA_SECRET_KEY=
ALPACA_PAPER_TRADE=true
# Data (optional)
POLYGON_API_KEY=
FINANCIAL_DATASETS_API_KEY=
# Master switch
TRADING_MODE=paper                  # paper | live
HUMAN_APPROVAL=true                 # require human approve before live orders
# Hard risk limits (RiskGuard)
MAX_POSITION_USD=2000
MAX_TRADE_PCT=2
DAILY_MAX_LOSS_USD=200
SAFETY_MODE_MAX_CONSEC_LOSSES=3    # NoFx-style auto-halt
MAX_OPTIONS_LEVEL=2
ALLOWED_SYMBOLS=AAPL,MSFT,NVDA,SPY
# Webhook (optional)
TRADINGVIEW_WEBHOOK_SECRET=
# Dashboard/API auth (NEVER ship no-auth — NoFx's mistake)
API_AUTH_TOKEN=
```

9. Deploy / install / run — full stack

Local (build + test):

```
git clone <repo> && cd ai-trader
cp .env.example .env          # [HUMAN] fill secrets
uv sync
pytest                        # Tier 0: unit (RiskGuard MUST pass)
python -m src.backtest.run    # Tier 1: backtests
TRADING_MODE=paper python -m src.main # Tier 2: paper loop
```

VPS (always-on brain — recommended):

```
# on Hetzner/D0 (~$5-6/mo), Docker installed:
git clone <repo> && cd ai-trader
# put secrets in host env / docker secrets
docker compose up -d          # api + worker (+ Postgres); restart: unless-stopped
```

Auto-deploy on push: a GitHub Action SSHes to the VPS and `git pull && docker compose up -d --build`.

Railway alternative: New project -> deploy from GitHub -> add a **Worker** service (not just web) -> set env vars -> enable restart-on-crash. ~\$5/mo hobby.

Vercel (dashboard + webhook only — NOT the loop): Import the GitHub repo, root = `dashboard/`, set `NEXT_PUBLIC_API_URL` to the VPS API + `API_AUTH_TOKEN`. Webhook route just forwards to the VPS.

10. Dry runs / test runs (correctness & accuracy)

- **Tier 0 — Unit (no network):** `pytest . test_risk.py` must prove RiskGuard + Safety Mode reject oversized/over-loss/off-hours/disallowed orders and halt after N losses.
- **Tier 1 — Backtest accuracy:** ≥ 3 -5 yrs history in VectorBT; cross-check a subset in `Backtesting.py` (engines should roughly agree); add slippage/commission; check for look-ahead bias; persist metrics. [HUMAN] **GATE**.
- **Tier 2 — Paper dry run:** `TRADING_MODE=paper`. Full agent loop vs Alpaca **paper**. Confirm orders appear in Alpaca paper dashboard, PnL updates, RiskGuard blocks bad orders, audit log captures every decision. Run **weeks**; compare paper PnL vs backtest. [HUMAN] **GATE**.
- **Tier 3 — Live, gated:** only after Tier 2; `TRADING_MODE=live`, `HUMAN_APPROVAL=true`, tiny size, low `MAX_*`, kill-switch reachable. [HUMAN] **EXPLICIT HUMAN SIGN-OFF**.
- **Observability:** structured logs + daily summary; optional `model-competition` runs two models on segregated paper accounts and compares decision logs before trusting either.

11. Cost table (build + paper phase, monthly)

Item	Ultra-low	Moderate
Broker (Alpaca)	\$0	\$0
Data	Alpaca + Polygon Free + FinDatasets free: \$0	Polygon Starter/Dev: \$29-79
LLM (Claude, cached/batched)	~\$5-20	~\$30-80
Hosting (VPS/Railway)	~\$5	\$20-40
Vercel dashboard	\$0	\$0
Total	~\$10-25/mo	~\$80-200/mo

12. Open knobs (the ask-tool failed earlier; defaults assumed — tell me to change any)

1. [KNOB] Standalone (assumed) vs. integrate-into-eigent vs. hybrid.
 2. [KNOB] Paper-first (assumed) vs. live-with-approval vs. autonomous.
 3. [KNOB] Broker: Alpaca (assumed) vs. Tradier vs. IBKR vs. TradingView-bridge.
 4. [KNOB] Budget tier: ultra-low (assumed) vs. moderate vs. flexible.
 5. [KNOB] Enable investor personas? model-competition (2nd model)? Polymarket sub-agent? FinRL RL module? (all optional, default off)
-

13. Sources

- Alpaca MCP: <https://github.com/alpacahq/alpaca-mcp-server> · <https://alpaca.markets/blog/alpaca-launches-mcp-server-v2/> · options: <https://docs.alpaca.markets/us/docs/options-trading>
- TradingAgents: <https://github.com/TauricResearch/TradingAgents> · <https://arxiv.org/pdf/2412.20138>
- ai-hedge-fund: <https://github.com/virattt/ai-hedge-fund>
- NoFx: <https://github.com/NoFxAiOS/nofx> · security caveat: <https://slowmist.medium.com/threat-intelligence-analysis-of-the-nofx-ai-automated-trading-vulnerability-e4f4664ad1e6>
- HKUDS/AI-Trader: <https://github.com/HKUDS/AI-Trader>
- FinRL / curated / prediction markets: <https://github.com/georgezouq/awesome-ai-in-finance> · <https://github.com/LLMQuant/awesome-trading-agents>
- TradingView webhooks/bridges: <https://www.tradingview.com/support/solutions/43000529348-how-to-configure-webhook-alerts/> · <https://blog.traderspost.io/article/automate-tradingview-ai-alerts-traderspost>
- Backtesting: <https://autotradelab.com/blog/backtrader-vs-nautilustrader-vs-vectorbt-vs-zipline-reloaded> · <https://bullalert.ai/blog/best-python-backtest-engines-2026>
- Hosting (Vercel limits / VPS / Railway): <https://vercel.com/kb/guide/what-can-i-do-about-vercel-serverless-functions-timing-out> · <https://www.nandann.com/blog/python-hosting-options-comparison> · <https://docs.railway.com/guides/cron-workers-queues>